

Basi di MatLab

Variabili

Per dichiarare una variabile possiamo scrivere, per esempio, `x = 10;`, e questa variabile viene salvata nel workspace, che è l'insieme delle variabili finora dichiarate, insieme ai loro valori. Questo è un linguaggio non fortemente tipato, il tipo viene dedotto automaticamente. Di default, tutte le variabili sono matrici. Quella `x` è una matrice di taglia 1×1 . Infatti, `size(x)` restituisce `1 1`.

Se chiamiamo `size(x)` senza assegnarlo a una variabile, il valore viene assegnato alla variabile `ans`. Possiamo eliminare tutte le variabili con `clear`, e mostrarle tutte con `whos`. Per vederne una sola, per esempio `x`, scriviamo `whos x`.

Salvare

Una funzionalità molto utile è quella di salvataggio del workspace, usando la funzione `save()`, che prende in input una stringa che sarà il nome del file, per esempio `save("workspace.mat")`. Potremo usare quel file per ripristinare il workspace con `load("workspace.mat")`.

Un'altra funzione utile è il *diary*, che permette di salvare tutto ciò che è successo nella sessione, quindi tutti i comandi eseguiti, sotto forma di file di testo. Il nome del file viene passato come parametro: `diary("diario.txt")`. Possiamo sospendere la registrazione con `diary off` e riprenderla con `diary on`.

Stampa e input

Il `;` non è obbligatorio alla fine di una riga. Di default i comandi stampano il risultato, il punto e virgola serve a impedire la stampa. Si impedisce perché è computazionalmente lenta. Non è l'unica opzione.

`disp()` permette di stampare il valore di singole variabili, come `disp(x)`, o di aggiungere una stringa, come `disp("Il valore di x è "+x)`.

La stampa formattata simile al C si fa con `fprintf()`: `fprintf("x = %d", x)`.
`%d %f \n`.

Per prendere valori in input si usa `input()`: `x = input("Inserisci un valore");`.

Polimorfismo

Certe funzioni sono polimorfiche, cioè cambiano il comportamento a seconda di come le chiamiamo. Per esempio, `eig(A)` restituisce lo spettro di A se assegnata a un vettore, e restituisce autovettori e autovalori in diagonale se assegnata a una coppia di matrici.

Comandi utili

Il comando `help` permette di consultare il manuale. Può prendere come parametro un comando, come `help diary`, e in quel caso stamperà la sezione che riguarda quel comando. Senza un parametro, fornisce una guida e un link alla documentazione.

`clc` elimina la cronologia della finestra dell'interfaccia a riga di comando.

Branching

Costrutti di scelta

Un'opzione per il branching è il costrutto `switch()` / `case` . Per il ramo da eseguire se il valore non corrisponde a nessuno dei casi si usa `otherwise` . Per terminare questo blocco, così come qualunque altro blocco, si usa `end` . Si possono ammassare più casi: `case {"c", "d"}` . Ripetere un caso, anche parzialmente, è un errore di sintassi, o un warning, a seconda della versione.

```
switch(x)
  case 1
    disp("Caso semplice")
  case {2, 3}
    disp("Caso ammassato")
  otherwise
    disp("Default")
end
```

Altrimenti, si usano `if()` ed `else()` . Il seguente codice è equivalente a quello sopra:

```
if(x==1)
  disp("Caso semplice")
elseif(x==2 | x==3)
  disp("Caso ammassato")
else
  disp("Default")
end
```

Costrutti iterativi

for opera su una variabile indice, di cui si deve specificare l'intervallo in cui itererà.
while invece controlla semplicemente che una condizione sia vera.

```
for i=1:10  
    disp(i)  
end
```

```
i=1  
while i<=10  
    disp(i)  
    i=i-1  
end
```

Funzioni semplici

Timer

È possibile utilizzare un timer per misurare il tempo di esecuzione di una porzione di codice, per esempio per confrontare la scalabilità di due algoritmi.

Il comando `tic` avvia il timer, e il comando `toc` lo ferma.

`toc` è polimorfico. Chiamato da solo, stamperà, per esempio, `Elapsed time is 0.002274 seconds.`, e non restituirà nessun valore da assegnare ad `ans`.

Altrimenti può essere assegnato a una variabile, come in `x = toc`, e in quel caso non stamperà niente.

Formati numerici

Il comando `format` prende un parametro per il formato della stampa dei numeri.

`format short` ci dà 4 cifre decimali. `format shorte` ci dà 4 cifre in notazione scientifica, similmente `long` e `longe` per 10 cifre. Questo comando non cambia in alcun modo i calcoli, solo la visualizzazione. L'effetto dura da quando viene chiamato in poi.

La funzione `fix()` tronca tutte le cifre decimali del parametro, quindi arrotonda verso 0. `floor()` arrotonda per difetto, verso $-\infty$. `ceil()` arrotonda per eccesso, verso $+\infty$.

`round()` arrotonda all'intero più vicino, e ha due parametri opzionali: il primo specifica la quantità di cifre dopo la virgola, e il secondo può solo essere una stringa, `"significant"`, che indica che deve considerare nel conteggio solo le cifre significative.

Creare matrici

Letterali

In MatLab appunto tutto è una matrice. `x=10` , `x` è una matrice 1×1 . `[1 2 3]` è un vettore riga, quindi una matrice 1×3 . Per fare più righe si usa il `;` : `[1 2 3; 4 5 6]` , le righe devono essere della stessa lunghezza. Quindi per fare un vettore colonna si fa `[1; 2; 3]` .

Matrici semplici

`eye(n)` genera la matrice identità di dimensione $n \times n$. `eye(m,n)` restituisce una matrice $m \times n$ che ha le prime m righe della matrice identità $n \times n$, o viceversa se $m > n$.

`zeros(n)` e `zeros(m,n)` sono simili ma per matrici contenenti solo 0, e `ones()` per matrici contenenti solo 1.

Casuali

`rand()` genera per numeri casuali tra 0 e 1 con distribuzione uniforme, `randn()` usa la distribuzione gaussiana con $\mu = 0$ e $\sigma = 1$.

`randi()` genera numeri interi casuali. Se il primo parametro è un vettore contenente due numeri, saranno usati come minimo e massimo. Se il primo parametro è un solo numero, sarà usato come massimo, e il minimo sarà 1.

Triangolari e diagonali

`tril(A)` estrae una matrice triangolare inferiore da A , mentre `triu(A)` estrae quella superiore. Viene mantenuta la diagonale.

`diag()` è una funzione polimorfica. Applicata a una matrice, estrae la diagonale e la mette in un vettore, e applicata a un vettore, genera una matrice diagonale che ha gli elementi di quel vettore nella diagonale.

Non possiamo generare una matrice diagonale direttamente, ma ci sono due modi per estrarla da una matrice qualunque: eliminare tutti gli elementi fuori dalla diagonale con `tril(triu(A))` o usare le due funzionalità di `diag()` con `diag(diag(A))` .

Altrimenti possiamo generare un vettore casuale e applicarci `diag()` .

Operazioni matriciali

Operazioni semplici

$A+B$ è la somma matriciale, elemento per elemento, e lo stesso vale per $A-B$. $A*B$ è il prodotto righe per colonne. A^2 è $A*A$. Il prodotto elemento per elemento si ottiene con $A.*B$. $A==B$ restituisce una matrice che ha 0 nelle posizioni in cui le due matrici sono diverse e 1 dove sono uguali (*maschera booleana*); lo stesso vale per $A<B$ e $A>B$.

Per calcolare l'inversa di una matrice, un modo è la funzione `inv()`. È una funzione un po' instabile, infatti $A*inv(A)$ alcuni elementi saranno molto vicini a 0, ma non esattamente 0, per esempio -0.000 . Un altro metodo è A^{-1} .

`det(A)` restituisce il determinante, `rank()` il rango.

La trasposta si ottiene con A' , semplicemente l'apostrofo. In realtà è l'aggiunta, ma l'aggiunta di una matrice reale è uguale alla trasposta.

Accedere agli elementi

Per accedere agli elementi di una matrice si usano le parentesi tonde, gli indici partono da 1. $A(1,1)$ è l'elemento in alto a sinistra.

Per mettere tutti gli elementi, letti colonna per colonna, in un vettore colonna si usa $A(:)$. Per selezionare tutta la seconda riga, $A(2,:)$. Per un intervallo, $A(1,2:3)$, "prima riga, dal secondo al terzo". Per posizionare un estremo alla fine della matrice, si usa `end`: $A(1,2:end)$. Si possono anche selezionare più righe e più colonne insieme, per esempio $A(2:3,1:2)$.

Dimensione

`size(A)` restituisce il numero di righe e il numero di colonne in un vettore.

`size(A,1)` restituisce solo il numero di righe, e `size(A,2)` restituisce il numero di colonne.

Norma

Per calcolare la norma di una matrice o di un vettore usiamo `norm()` . Per i vettori usa la norma euclidea, per le matrici la norma di Frobenius, di default. Per scegliere una norma specifica si usa il secondo parametro: `norm(A,1)` , `norm(A,2)` , `norm(A,inf)` .

Autovalori e autovettori

La funzione `eig()` è polimorfica: cambia il comportamento a seconda di come la si chiama. Se la si assegna a un vettore, restituisce lo spettro, se è assegnata a un vettore che contiene due matrici, assegna alla prima gli autovettori e alla seconda gli autovalori in diagonale. `diag()` converte da vettore a matrice in diagonale. Per trovare il raggio spettrale, si può usare `max(abs(eig()))` , perché `abs()` si applica elemento per elemento.

Grafici

`plot(x,y)` : vettore ascisse, vettore ordinate.

Esempio:

```
x = 0:0.1:10;  
y = cos(x);  
plot(x,y)
```

Possiamo anche aggiungere stile usando altri parametri:

```
plot(x,y, "LineWidth",3, "Color",[1,0.5,0.1], "LineStyle","--")
```

Linea di spessore 3, arancione, tratteggiata.

Per aggiungere un altro grafico insieme, passiamo altre coordinate, però mantiene lo stesso stile:

```
plot(x,sin(x),x,cos(x))
```

Per dare stili diversi, dobbiamo creare un altro grafico, chiamando di nuovo `plot()`.

Però lo mette in un riquadro separato. Per metterli insieme, usiamo `hold on`:

```
plot(...)  
hold on  
plot(...)
```

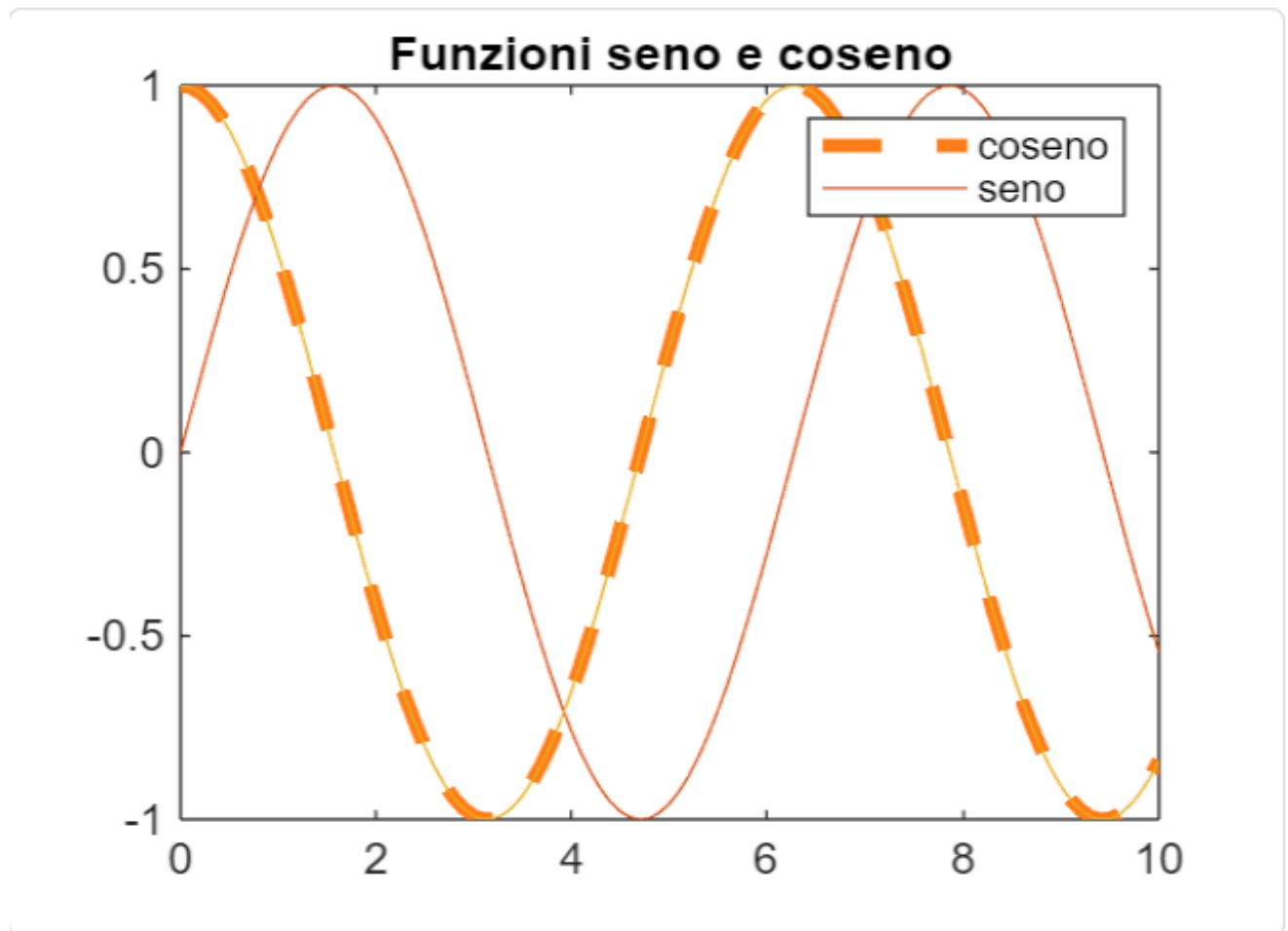
Possiamo anche mettere un titolo e una legenda:

```
title("Funzioni seno e coseno")  
legend("coseno", "seno")
```

Quindi, con questo codice:

```
x = 0:0.1:10;  
y = cos(x);  
plot(x,y, "LineWidth",3, "Color",[1,0.5,0.1], "LineStyle","--")  
hold on  
plot(x,sin(x),x,cos(x))  
title("Funzioni seno e coseno")  
legend("coseno", "seno")
```

Otteniamo questo grafico:



Definizione di funzioni

Ci sono tre metodi per definire delle funzioni.

Una funzione può essere definita in un file `.m`. Nell'intestazione, si specificano le variabili di cui viene restituito il valore, il nome della funzione e i parametri. La funzione termina con `end`. Per esempio:

```
function [y] = reciproco(x)
    y = 1/x;
end
```

Questa funzione assegna ad `y` il valore di $1/x$ e poi lo restituisce.

Alcuni parametri possono essere opzionali. Possiamo controllare il numero di parametri passati con `nargin`, e assegnare valori ai parametri non passati. Per esempio:

```
function [x, k] = gauss_seidel(A, b, x0, tau, kmax)
    % Parametri opzionali
    if nargin < 3 || isempty(x0)
        x0 = zeros(size(b));
    end
    if nargin < 4 || isempty(tau)
        tau = 1e-6;
    end
    if nargin < 5 || isempty(kmax)
        kmax = 1000;
    end
    ...
end
```

Si possono anche dichiarare dei puntatori a funzione, con `@`. Questo permette di creare alias, per esempio `recip = @reciproco`, ma un utilizzo più utile di `@` è la dichiarazione di funzioni anonime, che non richiedono la creazione di ulteriori file. La seguente funzione è equivalente a `reciproco()`:

```
rec = @(x) 1/x;
```

I puntatori a funzione possono essere passati ad altre funzioni come parametri.

Algoritmi

Trovare l'epsilon di macchina

Equivale a trovare un certo ε tale che $1 + \varepsilon > 1$.

```
e = 1;
while 1+e/2>1
    e = e/2;
end
disp("L'epsilon di macchina è "+e)
disp("L'epsilon secondo MatLab è "+eps)
```

Errore di cancellazione

Proviamo a fare un limite:

$$\lim_{x \rightarrow 0} \frac{1 - \cos(x)}{x^2} = \frac{1}{2}$$

```
for i=-1:-1:-10
    x = 10^i;
    y = (1-cos(x))/x^2
end
```

Se lo eseguiamo, il risultato sarebbe 0. Questo è dovuto all'errore di cancellazione:

$1 - \cos(x)$ diventa talmente piccolo da diventare 0 come numero di macchina. La soluzione è di riformulare la formula per evitare quel numero piccolo.

$$\lim_{x \rightarrow 0} \frac{1}{2} \left(\frac{\sin\left(\frac{x}{2}\right)}{\frac{x}{2}} \right)^2$$

```
for i=-1:-1:-10
    x = 10^i;
    y = 1/2*(sin(x/2)/(x/2))^2
end
```

Metodo di Gauss

L'algoritmo di Gauss permette di trasformare una qualunque matrice in una matrice triangolare superiore, in modo tale che i sistemi associati alle due matrici abbiano la stessa soluzione, cioè le matrici sono equivalenti.

Si annullano tutti gli elementi sotto la diagonale. Per farlo, si sottrae dalla riga che si sta modificando un'altra riga, moltiplicata per un certo coefficiente m . Il coefficiente è scelto in modo tale che l'elemento non nullo più a sinistra si annulli, quindi è il rapporto tra l'elemento non nullo più a sinistra delle due righe.

```
function [U,b] = gauss(A,b)

[n,m] = size(A);
if m~=n
    error("La matrice A deve essere quadrata");
end

if n~=size(b,1)
    error("La dimensione di b deve essere compatibile con A");
end

if abs(det(A)) < 1e-14
    error("La matrice A è singolare")
end

for k=1:n-1
    for j=k+1:n
        % Coefficiente per la riga
        m = A(j,k)/A(k,k);
        % Riga j <- Riga j - m*(Riga k)
        A(j,:)=A(j,:)-m*A(k,:);
        % Lo stesso, ma per il vettore
        b(j)=b(j)-m*b(k);
    end
end
U=A;
end
```

Con pivoting

L'algoritmo riscontra un problema quando il denominatore di m si annulla. Per risolvere questo problema, possiamo scambiare la riga in questione con un'altra riga in modo tale che l'elemento sulla diagonale non sia nullo, ottenendo un'altra matrice equivalente.

Questo si chiama *pivoting*, e l'elemento diagonale è detto elemento pivot. Per questioni di stabilità numerica, è sempre meglio scambiare con la riga che ha l'elemento massimo, per ottenere la frazione più piccola possibile.

```
function [U,b] = gauss_pivot(A,b)
    [n,m] = size(A);
    if m~=n
        error("La matrice A deve essere quadrata");
    end
    if n~=size(b,1)
        error("La dimensione di b deve essere compatibile con A");
    end
    if abs(det(A)) < 1e-14
        error("La matrice A è singolare")
    end
    for k=1:n-1
        % Ricerca della riga con elemento massimo
        [~,pos]=max(abs(A(k:n,k)));
        l = pos + k - 1;
        % Scambio
        A([k l],:) = A([l k],:);
        b([k l]) = b([l k]);
        % Metodo di Gauss normale
        for j=k+1:n
            m = A(j,k)/A(k,k);
            A(j,:)=A(j,:)-m*A(k,:);
            b(j)=b(j)-m*b(k);
        end
    end
    U=A;
end
```

Fattorizzazione ALU

Una matrice quadrata può essere espressa sotto forma di prodotto di due matrici triangolari, di cui una inferiore e una superiore: $A = LU$. Per trovare U si usa il metodo di Gauss. L è invece definita come segue:

$$l_{ij} = \begin{cases} m_{ij} & i > j \\ 1 & i = j \\ 0 & j > i \end{cases}$$

Dove m_{ij} è il coefficiente per la riga j e la colonna k trovato nel metodo di Gauss.

Questa fattorizzazione si può usare per risolvere un sistema. Avendo $Ax = b$, lo posso riscrivere come $LUx = b$, definire il vettore $y = Ux$ e ottenere un sistema:

$$\begin{cases} Ly = b \\ Ux = y \end{cases}$$

```
function [L,U]=alu(A)
    [n,m] = size(A);
    if m~=n
        error("La matrice A deve essere quadrata");
    end
    if abs(det(A)) < 1e-14
        error("La matrice A è singolare")
    end
    n=size(A,1);
    L = eye(n);
    for k = 1:n-1
        for j = k+1:n
            % Metodo di Gauss
            m = A(j,k)/A(k,k);
            A(j,:)=A(j,:)-m*A(k,:);
            % Gli elementi di L sono i coefficienti
            L(j,k)=m;
        end
    end
    U=A;
end
```

PALU

Una variante in cui si aggiunge una matrice P , che è una matrice di permutazione, cioè una matrice ottenuta tramite una serie di scambi dalla matrice identità. Data una matrice A , PA è la matrice ottenuta eseguendo gli stessi scambi su A . Nel caso di questo algoritmo, P corrisponde agli scambi eseguiti nel pivoting.

```
function [L,U,P] = palu_decomposition(A)
    [n,m] = size(A);
    if m~=n
        error("La matrice A deve essere quadrata");
    end
    if abs(det(A)) < 1e-14
        error("La matrice A è singolare")
    end
    P=eye(n);
    L=eye(n);
    for k=1:n-1
        % Ricerca della riga con elemento massimo
        [~,pos]=max(abs(A(k:n,k)));
        l = pos + k - 1;
        % Scambio
        A([k l], :) = A([l k], :);
        P([k l], :) = P([l k], :);
        % Bisogna modificare solo gli elementi sotto la diagonale
        L([k,l], 1:k-1) = L([l,k], 1:k-1);
        for j=k+1:n
            % Comportamento normale del metodo di Gauss
            m = A(j,k)/A(k,k);
            A(j,:)=A(j,:)-m*A(k,:);
            % Assegnazione di L
            L(j,k)=m;
        end
    end
    U=A;
end
```


Gauss-Seidel e Jacobi

Entrambe le funzioni prendono in input A e b , il vettore di partenza x_0 , la tolleranza τ , e k_{max} , il numero massimo di iterazioni dopo il quale assumiamo che non converga.

Controlliamo il raggio spettrale per sapere subito se non converge.

Per sapere se siamo arrivati alla convergenza, controlliamo se l'errore relativo tra due vettori è minore di un certo numero molto piccolo, quello che noi abbiamo chiamato τ . Potremmo controllarlo con $\text{norm}(x_{\text{new}} - x_0) / \text{norm}(x_0) > \tau$, ma al denominatore potremmo avere 0, quindi moltiplichiamo a destra e a sinistra e otteniamo $\text{norm}(x_{\text{new}} - x_0) > \tau * \text{norm}(x_0)$.

Usciti dal ciclo, segnaliamo all'utente se ci sono stati problemi.

Gauss-Seidel

La funzione ricorsiva ha $B = (D - E)^{-1}F$ e $f = (D - E)^{-1}b$.

```
function [x, k] = gauss_seidel(A, b, x0, tau, kmax)
    % Parametri opzionali
    if nargin < 3 || isempty(x0)
        x0 = zeros(size(b));
    end
    if nargin < 4 || isempty(tau)
        tau = 1e-6;
    end
    if nargin < 5 || isempty(kmax)
        kmax = 1000;
    end
    % Split additivo
    D = diag(diag(A));
    E = -tril(A,-1); % parametro per non prendere la diagonale
    F = -triu(A,1); % il parametro cambia segno
    B = (D-E)\F;
    f = (D-E)\b;

    rho = max(abs(eig(B)));
    if rho >= 1
        error("Il metodo non converge, raggio spettrale %.4f", rho)
    end
    flag = true;
    k = 0;
    x_new = x0;
    while flag
        x0 = x_new;
        k = k+1;
        x_new = B*x0+f;
        flag = k<kmax && norm(x_new-x0)>tau*norm(x0);
    end
    if k >= kmax
        error("Il metodo non converge entro %d iterazioni", kmax)
    end
    x = x_new;
end
```

Jacobi

L'algoritmo è uguale, ma con $B = D^{-1}(E + F)$ e $f = D^{-1}b$.

```
function [x, k] = jacobi(A, b, x0, tau, kmax)
    % Parametri opzionali
    if nargin < 3 || isempty(x0)
        x0 = zeros(size(b));
    end
    if nargin < 4 || isempty(tau)
        tau = 1e-6;
    end
    if nargin < 5 || isempty(kmax)
        kmax = 1000;
    end
    % Split additivo
    D = diag(diag(A));
    E = -tril(A,-1);
    F = -triu(A,1);
    B = D\(E+F);
    f = D\b;

    rho = max(abs(eig(B)));
    if rho >= 1
        error("Il metodo non converge, raggio spettrale %.4f", rho)
    end
    flag = true;
    k = 0;
    x_new = x0;
    while flag
        x0 = x_new;
        k = k+1;
        x_new = B*x0+f;
        flag = k<kmax && norm(x_new-x0)>tau*norm(x0);
    end
    if k >= kmax
        error("Il metodo non converge entro %d iterazioni", kmax)
    end
    x = x_new;
end
```

Sistemi non lineari

Utilizziamo metodi iterativi per trovare le radici dei sistemi non lineari.

Metodo di bisezione

Questo algoritmo ha cinque parametri: la funzione di cui vogliamo una radice, gli estremi a e b dell'intervallo nel quale si trova la radice che vogliamo trovare, una tolleranza detta τ e il numero massimo di iterazioni, k_{max} .

Assumendo che la funzione sia continua, per il teorema di Bolzano-Weierstrass, se i valori della funzione in a e in b sono discordi, allora è sicuro che una radice si trovi in quell'intervallo. In caso contrario, l'algoritmo genera un errore.

Un'ordinata minore o uguale a τ è considerata nulla.

Ad ogni iterazione, il punto medio di $[a, b]$, detto c , viene scelto come nuovo estremo, sostituendo quello concorde a c . Alla fine dell'algoritmo, c è la radice.

```

function [x,k] = bisec(fun, a, b, tau, kmax)
    % Controllo degli estremi e del punto medio
    fa = fun(a);
    fb = fun(b);
    if fa*fb> 0
        error("Non è garantito che ci sia una radice nell'intervallo")
    end
    c = (a+b)/2;
    fc = fun(c);
    if abs(fc)<tau
        x = c;
        return;
    end
    % Iterazioni
    flag = 1;
    k = 0;
    while flag
        k = k+1;
        if fa*fc<0
            b = c;
            fb = fc;
        else
            a = c;
            fa = fc;
        end

        c = (a+b)/2;
        fc = fun(c);

        flag = (abs(a-b)>tau) && abs(fc)>tau && (k<kmax);
    end
    if(k>=kmax) % Superato il numero massimo di iterazioni
        x = inf;
        warning("Non è stata trovata una soluzione");
    elseif(abs(fc)<tau) % Trovata una radice
        x = c;
    else % I due estremi sono uguali
        x = (a+b)/2;
    end
end
end

```

Metodo di Newton

L'approccio è un po' diverso: si parte da un singolo valore, x_0 , e lo si aggiorna iterativamente con la formula:

$$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}$$

Ci si ferma se l'ordinata è minore della tolleranza, se il valore di x nelle due iterazioni è cambiato poco, o se si è superato il numero massimo di iterazioni.

Ci si ferma se:

- L'ordinata è minore o uguale alla tolleranza, $f(x) \leq \tau$;
- Il valore di x è cambiato poco nelle ultime due iterazioni, $\frac{|x^{(k+1)} - x^{(k)}|}{|x^{(k)}|} \leq \tau$;
- Si è superato il numero massimo di iterazioni.

Questo metodo non si può usare se la derivata si annulla in x_0 , o in qualunque $x^{(k)}$ durante l'esecuzione.

I parametri sono la funzione di cui vogliamo trovare la radice, la derivata di quella funzione, x_0 , τ e k_{max} .

```
function [x,k] = newton(fun, fund, x0, tau, kmax)
    k = 0;

    f0 = fun(x0);
    if abs(f0)<tau
        x = x0;
        return;
    end

    fd = fund(x0)
    if abs(fd)<1e-100
        error("La derivata in x0 si annulla")
    end

    k = 1;
    x_new = x0 - (f0/fd);
    flag = 1;
```

```

while flag
    k = k+1;
    xk = x_new;
    fk = fun(xk);
    fd = fund(xk);

    if abs(fd)<1e-100
        error("La derivata si annulla")
    end

    x_new = xk - (fk/fd);

    % Per stabilità numerica, moltiplichiamo anziché dividere
    flag = (abs(fun(x_new))>tau) && (abs(x_new-xk)>abs(xk)*tau)
end

if k>kmax
    warning("troppe iterazioni")
    x = inf;
else
    x = x_new;
end
end

```

Questo metodo ha delle varianti. Generalizzando la formula come

$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{m_k}$, possiamo considerare il metodo di Newton come un caso specifico in cui $m_k = f'(x^{(k)})$. Possiamo quindi derivarne altri metodi variando la formula da cui otteniamo m_k .

Metodo delle secanti

Il metodo delle secanti è una variante del metodo di Newton in cui

$$m_k = \frac{f(x^{(k+1)}) - f(x^{(k)})}{x^{(k+1)} - x^{(k)}}.$$

Non c'è più bisogno di passare la derivata della funzione come parametro, però ci sono due valori di partenza come il metodo di bisezione.

```
function [x,k] = secanti(fun, x0, x1, tau, kmax)
    k = 0;

    f0 = fun(x0);
    if abs(f0)<tau
        x = x0;
        return;
    end

    f1 = fun(x1);
    if abs(f1)<tau
        x = x1;
        return;
    end

    % Usiamo il reciproco di m per non dividere due volte
    mrec = (x1-x0)/(f1-f0);
    if abs(f1-f0)<1e-100
        error("Il denominatore si annulla")
    end

    k = 1;
    x_new = x1 - (f1*mrec);
    flag = 1;

    while flag
        k = k+1;
        x0 = x1;
        x1 = x_new;
        f0 = fun(x0);
        f1 = fun(x1);
```



```

mrec = (x1-x0)/(f1-f0);
if abs(f1-f0)<1e-100
    error("Il denominatore si annulla")
end

x_new = x1 - (f1*mrec);

% Per stabilità numerica, moltiplichiamo anziché dividere
flag = (abs(fun(x_new))>tau) && (abs(x_new-x1)>abs(x1)*tau
end

if k>kmax
    warning("troppe iterazioni")
    x = inf;
else
    x = x_new;
end
end

```

Metodo delle corde

In questo caso, il valore di m è scelto a priori e non cambia tra un'iterazione e l'altra. Deve quindi essere passato come parametro.

```
function [x,k] = corde(fun, m, x0, tau, kmax)
    k = 0;
    f0 = fun(x0);
    if abs(f0)<tau
        x = x0;
        return;
    end
    if abs(m)<1e-100
        error("Il denominatore si annulla")
    end
    k = 1;
    x_new = x0 - (f0/m);
    flag = 1;
    while flag
        k = k+1;
        x0 = x_new;
        f0 = fun(x0);
        x_new = x0 - (f0/m);
        % Per stabilità numerica, moltiplichiamo anziché dividere
        flag = (abs(fun(x_new))>tau) && (abs(x_new-x0)>abs(x0)*tau)
    end
    if k>kmax
        warning("troppe iterazioni")
        x = inf;
    else
        x = x_new;
    end
end
```